

## CHAPTER – VIII

### STRINGBUFFER AND STRINGBUILDER:

- StringBuffer and StringBuilder are used to create and modify String efficiently.
- String is immutable.

**STRINGBUFFER** – Multi – thread environment.

**STRINGBUILDER** – Single – thread environment(most cases).

### STRINGBUFFER:

- StringBuffer is a predefined class in java used to create and mutable(modifiable) strings.
- It is also thread – safe, because all its methods are synchronized.

### EXAMPLE:

```
public class StringBufferExample {  
    public static void main(String[] args) {  
        StringBuffer sb = new StringBuffer("Hello");  
        sb.append(" World");  
        System.out.println("Append: " + sb);  
        sb.insert(5, " Java");  
        System.out.println("Insert: " + sb);  
        sb.replace(6, 10, "C++");  
        System.out.println("Replace: " + sb);  
        sb.delete(6, 10);  
    }  
}
```

```
        System.out.println("Delete: " + sb);
        sb.reverse();
        System.out.println("Reverse: " + sb);
    }
}
```

### **STRINGBUILDER:**

- StringBuilder is used to create mutable strings.
- It is not synchronized.

### **EXAMPLE:**

```
public class StringBuilderExample {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Hello");
        sb.append(" World");
        System.out.println("Append: " + sb);
        sb.insert(5, " Java");
        System.out.println("Insert: " + sb);
        sb.replace(6, 10, "C++");
        System.out.println("Replace: " + sb);
        sb.delete(6, 10);
        System.out.println("Delete: " + sb);
        sb.reverse();
        System.out.println("Reverse: " + sb);
    }
}
```

}

### **THREAD:**

- Thread is the smallest unit of program that can run independently.
- It allows multiple tasks at the same time.

### **EXAMPLE:**

```
class MyThread extends Thread{  
    public void show(){  
        System.out.println("Thread is running");  
    }  
}
```

### **THREAD – SAFE:**

- A code or object is thread – safe if multiple threads can use it at the same time without problems.

### **MULTITHREADING:**

- Multithreading is the process of executing two or more threads at the same time within a single java program.

### **ADVANTAGES:**

- Improves Performance
- Better CPU utilization
- Useful for real time Applications
- Allows background tasks

**SYNCHRONIZATION:**

- Synchronization means only one thread is allowed to use a method or block of code at a time.

**EXAMPLE:**

```
synchronized void print(){  
    System.out.println("Hello");  
}
```